Introduction to Database Systems Spring 2025

Lecture 2 - Designing Databases

Omar Shahbaz Khan

TODO

- Entity-Relationship (ER) Models / Diagrams
  - Entities and Attributes
  - Relationships
  - Cardinalities
  - Partial Relationship Keys
  - Weak Entities
- (Enhanced) ER Diagram
  - Aggregation
  - Generalization/Specialization
  - Categorization
- Translation to SQL DDL

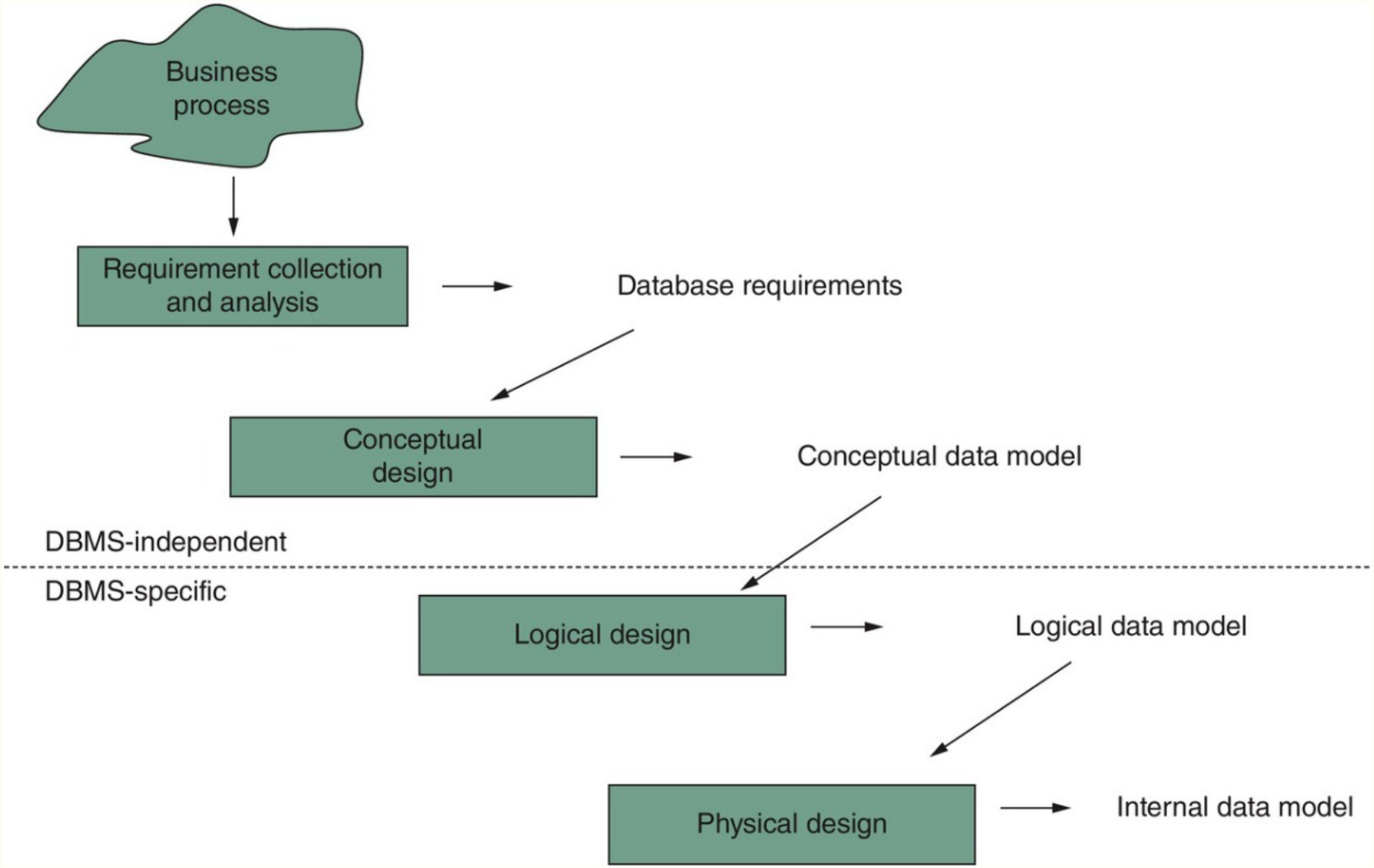
Design Process

Not a focus of this course

Sketch the components in an ER model

Convert ER model to SQL DDL

Later in the course



Question

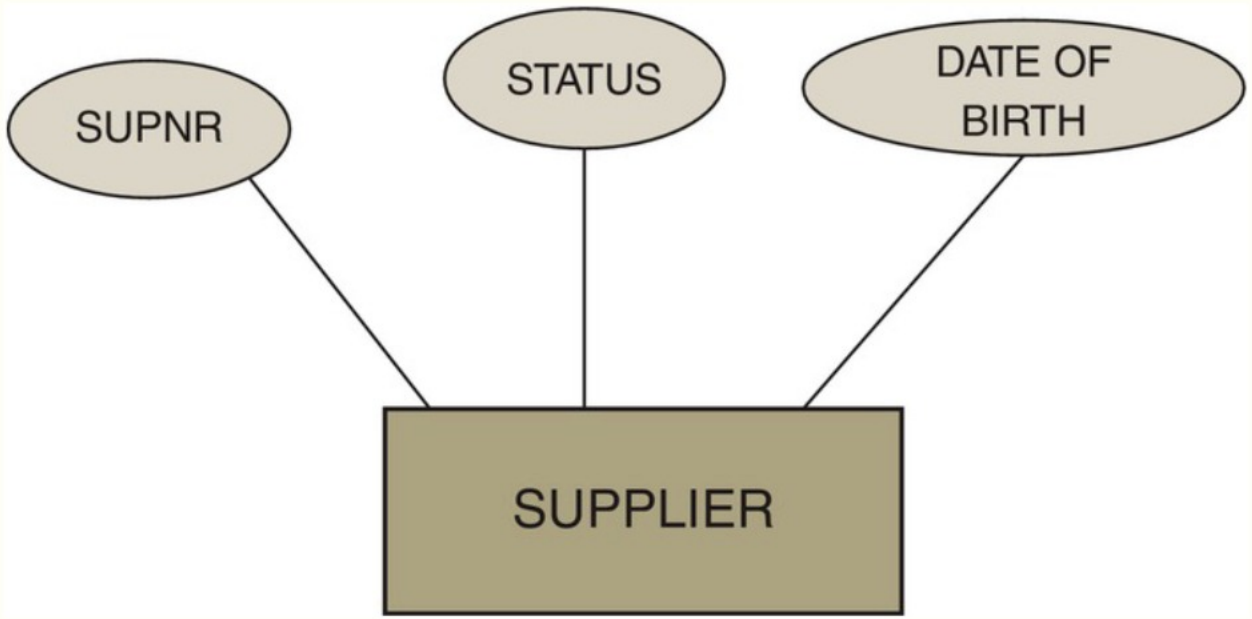
- Given a list of requirements for a database:
  - Why is it important to start by drawing a diagram?
  - Why can't we just start creating the database and the tables?

Entity-Relationship Model / Diagram

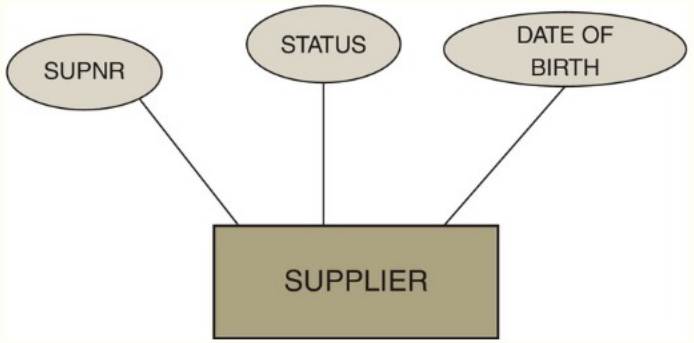
- Conceptual Data Modeling technique was defined by Peter Chen (1976)
- ER = modeling concepts + visual representation
  - ER/EER notation is not standardized
  - Every textbook/company/tool has its own visual representation
  - Core concepts are universally accepted
  - EER vs ER rarely distinguished -> we just call it ER!
- UML can be used as design notation
  - Slight differences
  - No UML in this course
- Focus on ER notation from the book
  - … plus, some minor extensions – made clear in the lectures
  - In exercises, project, exam: Use the notation in book, lectures

Entity and Attribute Types

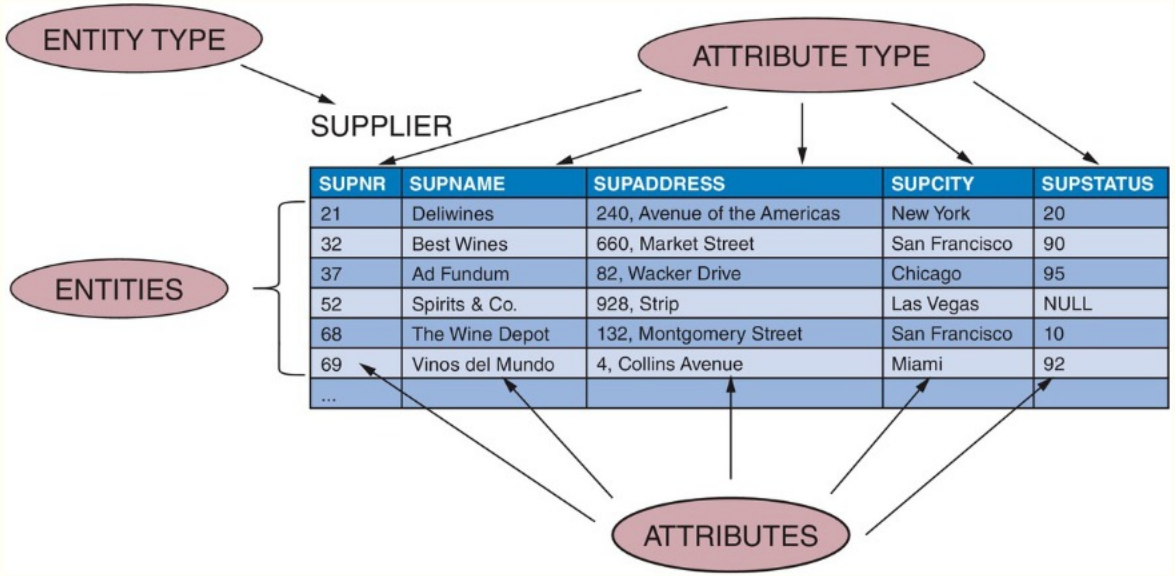
- Entity Type:
  - Set of similar "things" / business concept
    - Movie, Vehicle, Course, Supplier, etc.
  - Entity = Instance:
    - Movie: Interstellar, 2014
- Attribute Type:
  - Describes one aspect of an entity set
    - Movie: title, year



Entity and Attribute Types (DDL)

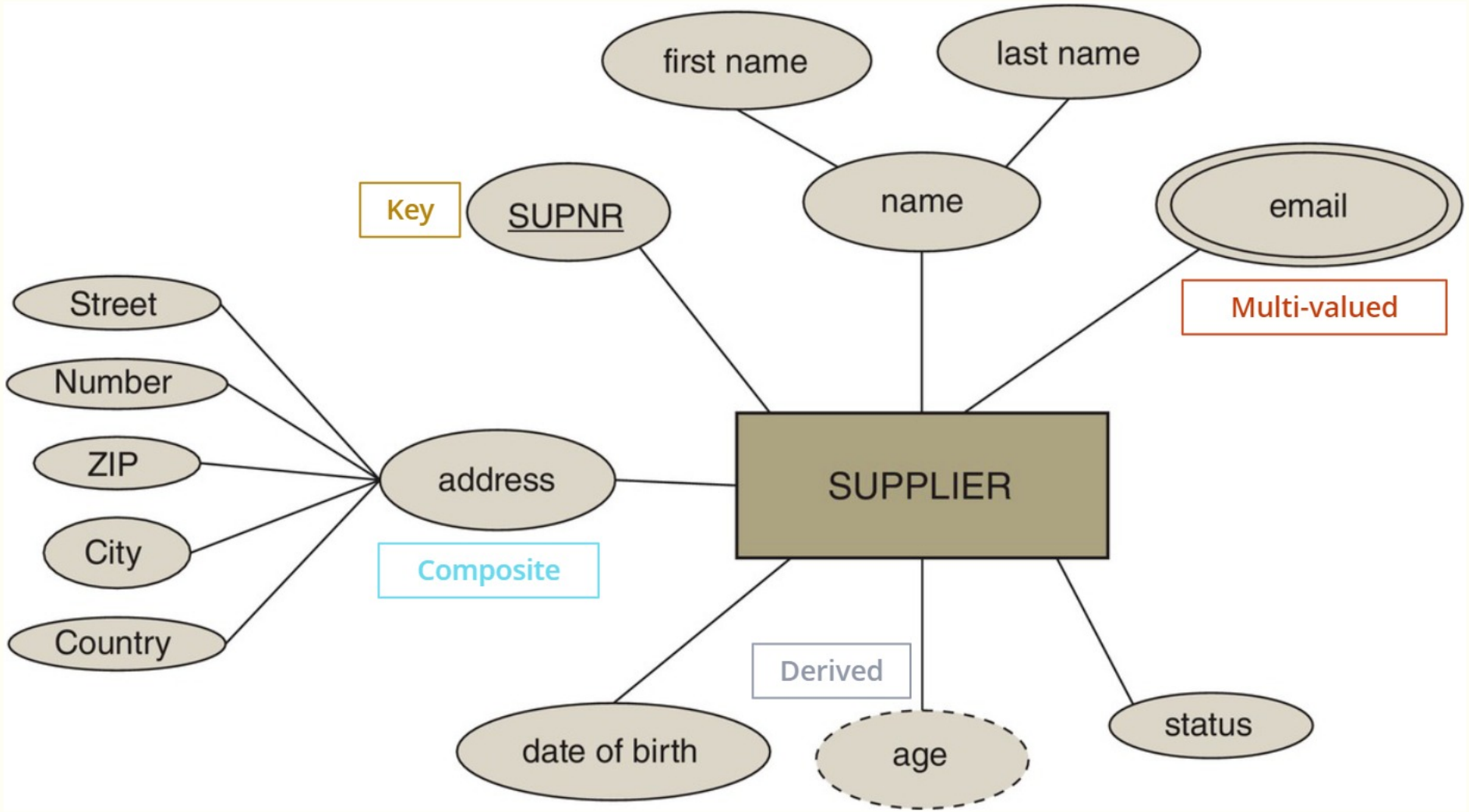



```sql
CREATE TABLE Supplier (
    SupNr INT NOT NULL,
    SupStatus INT,
    SupCity VARCHAR(50) NOT NULL,
    ...
);
```



SUPNR	SUPNAME	SUPADDRESS	SUPCITY	SUPSTATUS
21	Deliwines	240, Avenue of the Americas	New York	20
32	Best Wines	660, Market Street	San Francisco	90
37	Ad Fundum	82, Wacker Drive	Chicago	95
52	Spirits & Co.	928, Strip	Las Vegas	NULL
68	The Wine Depot	132, Montgomery Street	San Francisco	10
69	Vinos del Mundo	4, Collins Avenue	Miami	92
...				

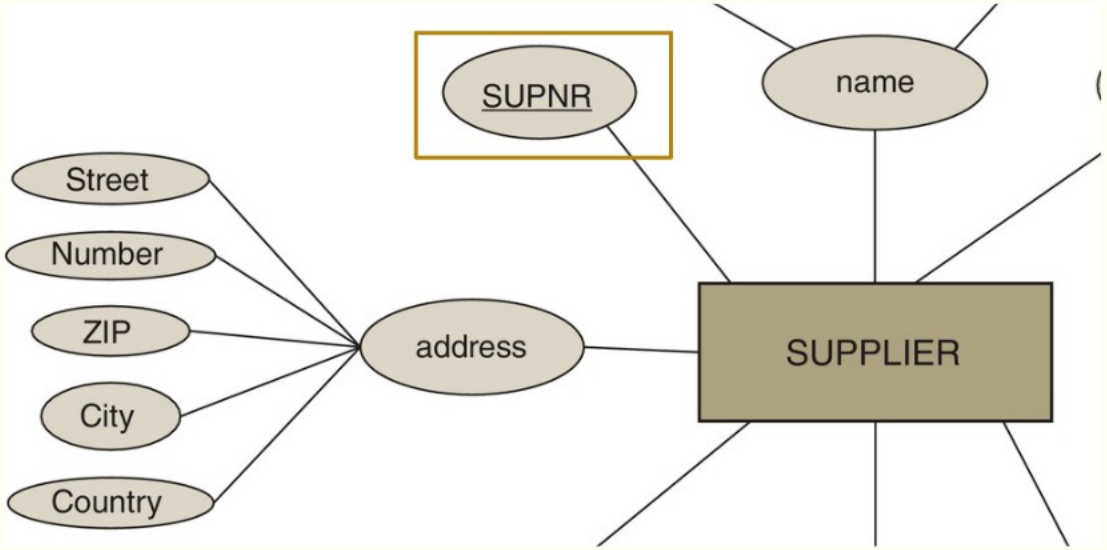
Different Attribute Types



Key Attribute

- Underlined = PRIMARY KEY
- ER Models cannot show candidate keys
  - They must be noted somewhere else!
  - They must still be UNIQUE in the SQL DDL

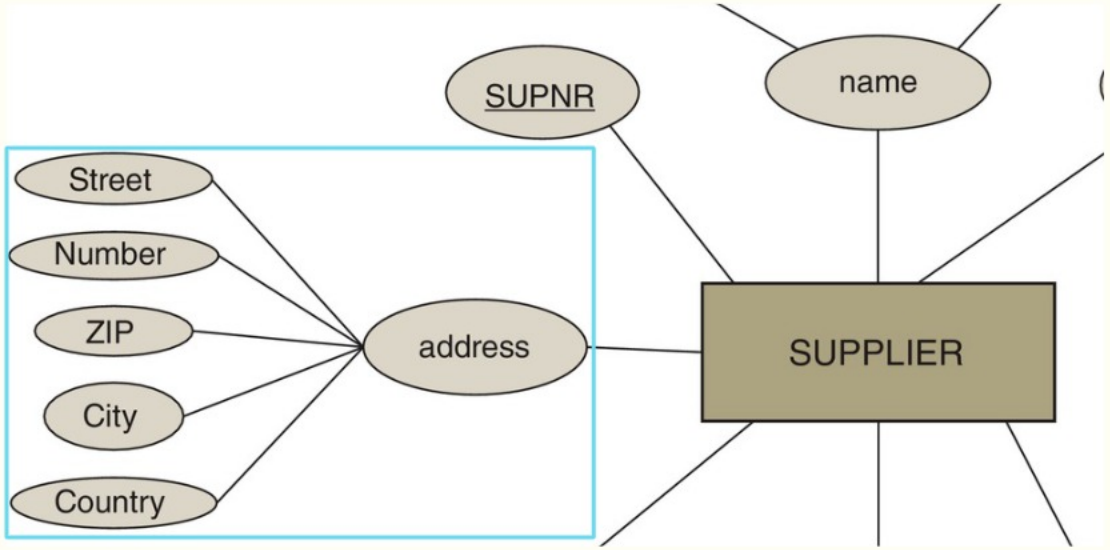
```
CREATE TABLE Supplier (
   SupNr INT PRIMARY KEY,
   ...
);
```



Composite Attribute

- Grouping of related attributes
- In DDL we just add the related attributes, not the grouping

```
CREATE TABLE Supplier (
   ...
   Street VARCHAR(50) NOT NULL,
   Number INT NOT NULL,
   ZIP INT NOT NULL,
   City VARCHAR(50) NOT NULL,
   Country VARCHAR(50) NOT NULL,
   ...
);
```

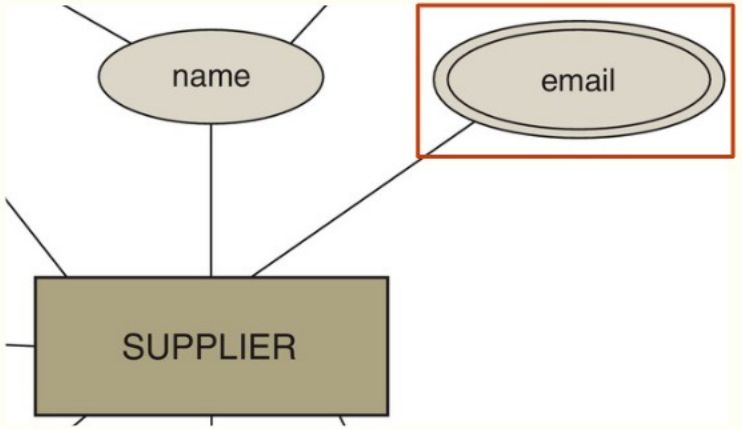


- **Why is this useful?**

Multi-valued Attribute

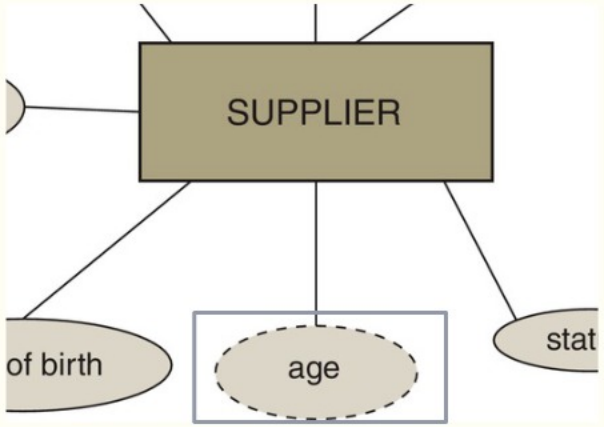
- Attribute that is allowed to have more than one value
- Create a table for the attribute

```
CREATE TABLE Email (
   Email VARCHAR(50),
   SupNr INT REFERENCES Supplier(SupNr),
   PRIMARY KEY (Email, SupNr)
);
```

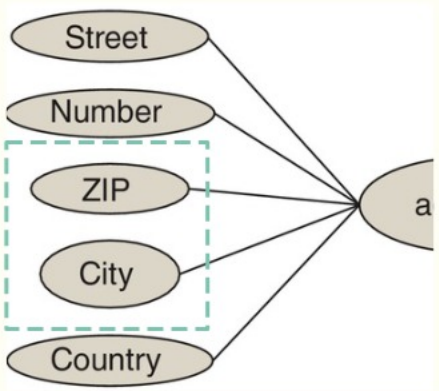


Derived Attribute

- Not discussed in the PDBM book!
- Option 1: Create an attribute and maintain it
  - Either with a database trigger, or regular update process
- Option 2: Create a view that computes it
  - Views are virtual tables using predefined queries



- Neither option is great, best to avoid such cases at the database levels
- Sometimes there is a second kind of inter-attribute relationship
  - Here: ZIP ⟶ City
  - Called functional dependencies (FDs)
  - ER diagrams may miss such relationships!
  - We fix this with normalization (next lecture)



Practice (2 min)

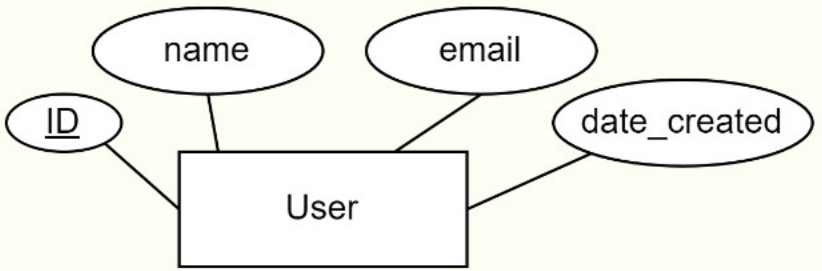
- We are developing an app, and one of the requirements are the following:
  - The app has users who each have an ID, name, a unique e-mail, and the date they joined / the user was created

- Draw an ER Diagram that matches this requirement
  - You can use pen and paper/tablet/phone, Paint, draw.io, etc.

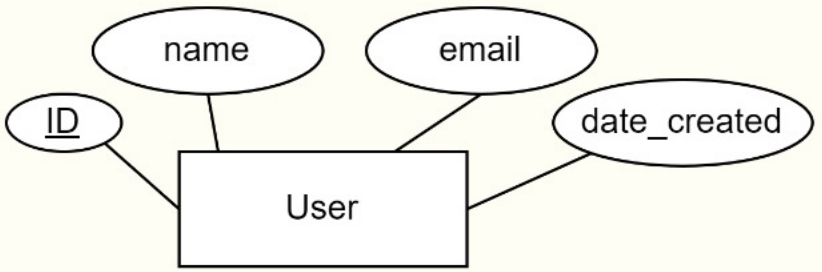
Practice (2 min)

- We are developing an app, and one of the requirements are the following:
  - The app has users who each have an ID, name, a unique e-mail, and the date they joined / the user was created

- Draw an ER Diagram that matches this requirement
  - You can use pen and paper/tablet/phone, Paint, draw.io, etc.



Question

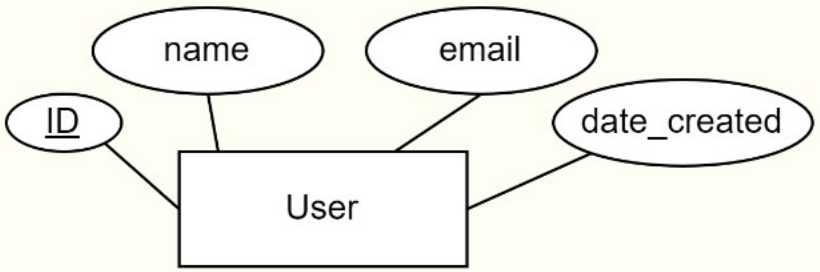


- Which of the following DDL matches this diagram?

```
CREATE TABLE User (
   ID INT NOT NULL,
   name VARCHAR(50) NOT NULL,
   email VARCHAR(50) UNIQUE NOT NULL,
   date_created DATE NOT NULL
);
```

```
CREATE TABLE User (
   ID INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   email VARCHAR(50) UNIQUE NOT NULL,
   date_created DATE NOT NULL
);
```

Question



- Which of the following DDL matches this diagram?

```
CREATE TABLE User (
   ID INT NOT NULL,
   name VARCHAR(50) NOT NULL,
   email VARCHAR(50) UNIQUE NOT NULL,
   date_created DATE NOT NULL
);
```

```
CREATE TABLE User (
   ID INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   email VARCHAR(50) UNIQUE NOT NULL,
   date_created DATE NOT NULL
);
```

Check Constraints

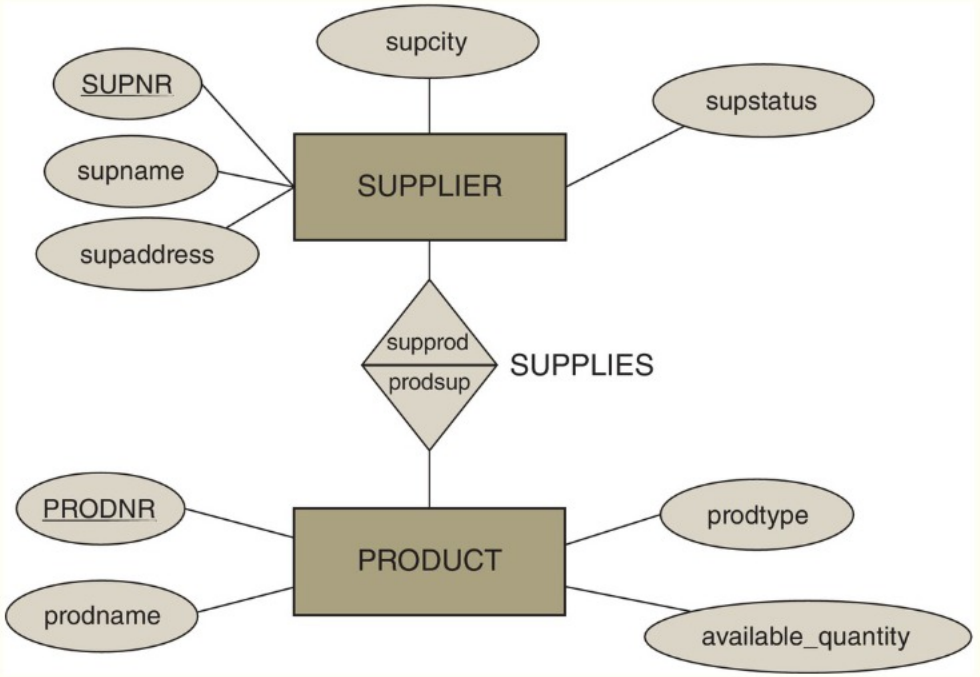
- Requirement specifies constraints related to the value of an attribute type or correlation between multiple attribute types
  - salary cannot exceed 300000
  - price cannot be negative
  - status must be "active" or "inactive"
  - allowance has to be lower than salary
- For simple validation constraints we can use CHECK statements in DDL

```
CREATE TABLE Test (
   salary FLOAT NOT NULL CHECK (salary < 300000),
   price FLOAT NOT NULL,
   status VARCHAR(8) NOT NULL CHECK (status IN ('active', 'inactive')),
   allowance FLOAT NOT NULL CHECK (allowance < salary),
   CHECK (price >= 0)
);
```

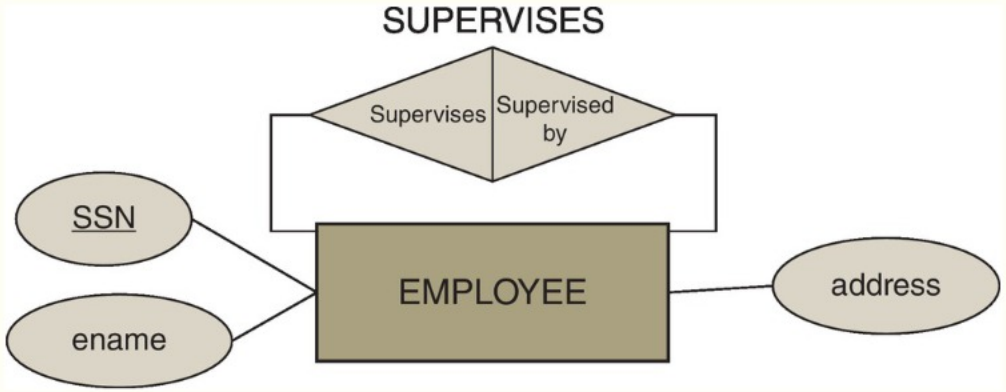
- For more complex requirements we write them down and deal with them using functions or in application logic

Relationship Types

- Relate two or more entities (with roles)
- Roles may be omitted when obvious



- Unary relationship type:
  - Relationship between two instances of the same entity type



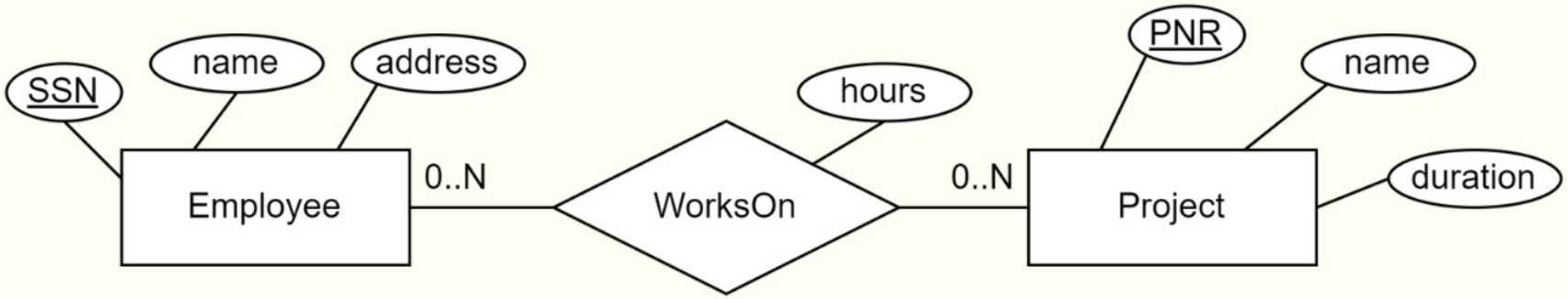
Relation vs Relationship Type

- Relation (relational model)
  - set of tuples
- Relationship type (ER model)
  - describes relationship between entities

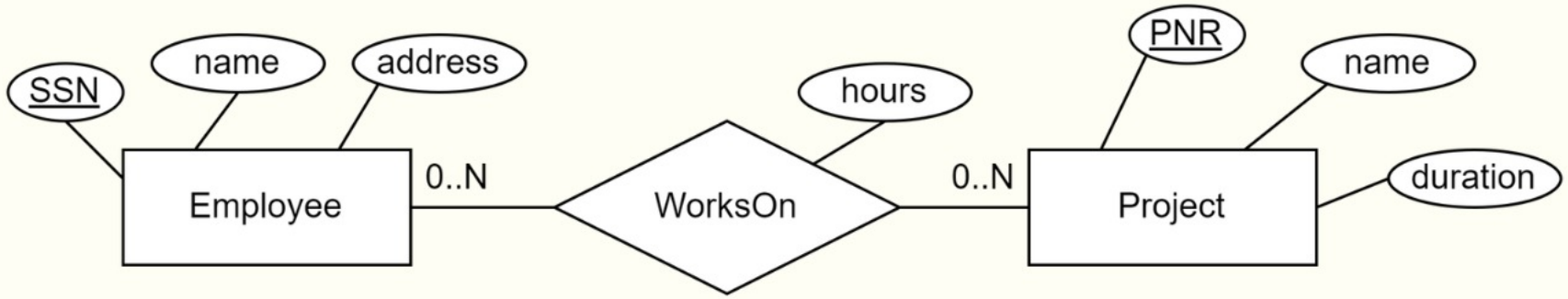
- Both entity types and relationship types (from an ER model) will be represented by relations (in the relational model)

Relationship Attribute Types

- Relationships can also have attribute types

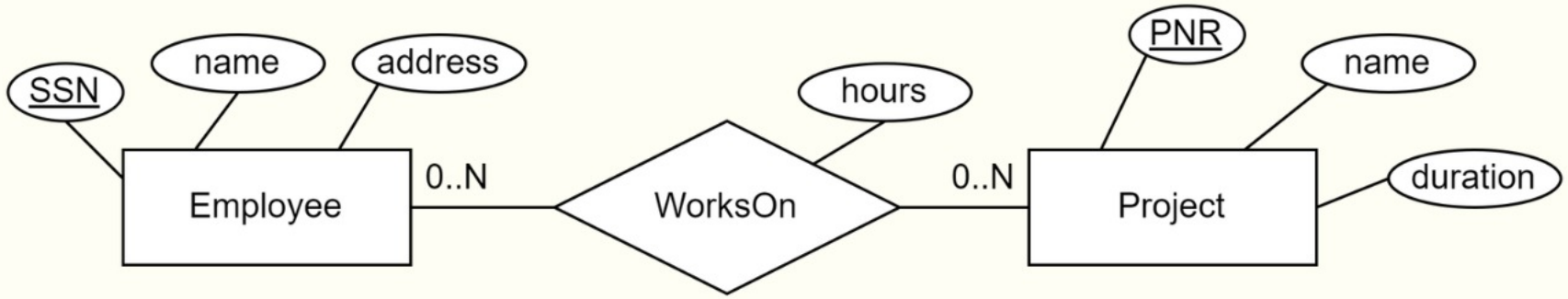


Basic Relationship Table in SQL DDL




```sql
CREATE TABLE WorksOn (
   EmployeeID INT,        -- role (key of Employee)
   ProjectID INT,         -- role (key of Project)
   hours INT NOT NULL,    -- attribute
   PRIMARY KEY (EmployeeID, ProjectID),
   FOREIGN KEY (EmployeeID) REFERENCES Employee (SSN),
   FOREIGN KEY (ProjectID) REFERENCES Project (PNR)
);
```

Basic Relationship Table in SQL DDL

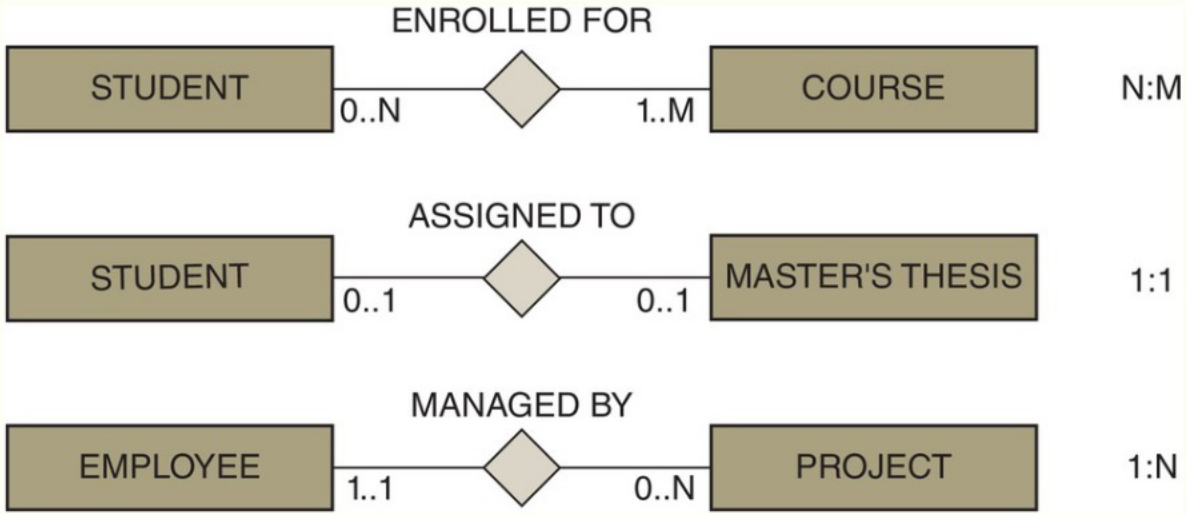


Alternative way to reference foreign keys:

```
CREATE TABLE WorksOn (
   EmployeeID INT REFERENCES Employee (SSN),
   ProjectID INT REFERENCES Project (PNR),
   hours INT NOT NULL,
   PRIMARY KEY (EmployeeID, ProjectID)
);
```

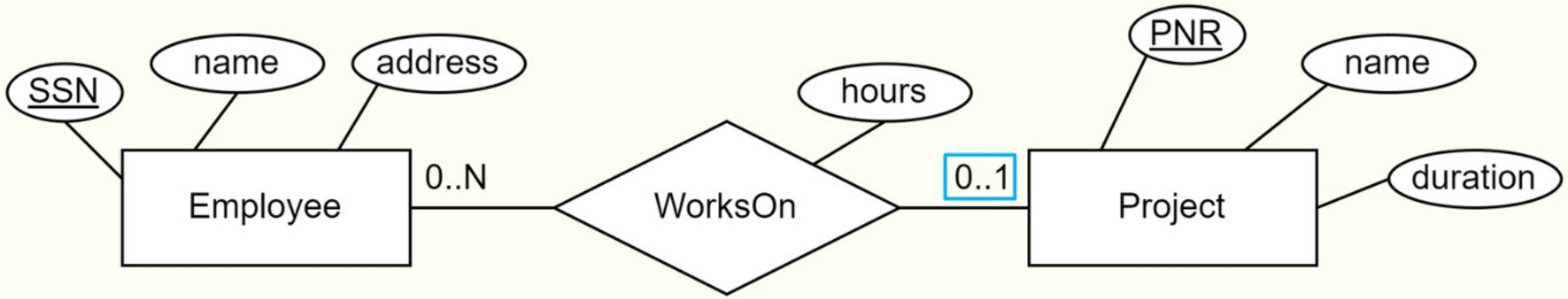
Cardinalities

- Relationships always have cardinalities
  - Minimum: 0 or 1
  - Maximum: 1 or N / M / L / * / ...
- Read:
  - Entity (ignore) Relationship Cardinality Entity
  - Student has to enroll in at least one course



- Cardinalities impact the resulting table structure

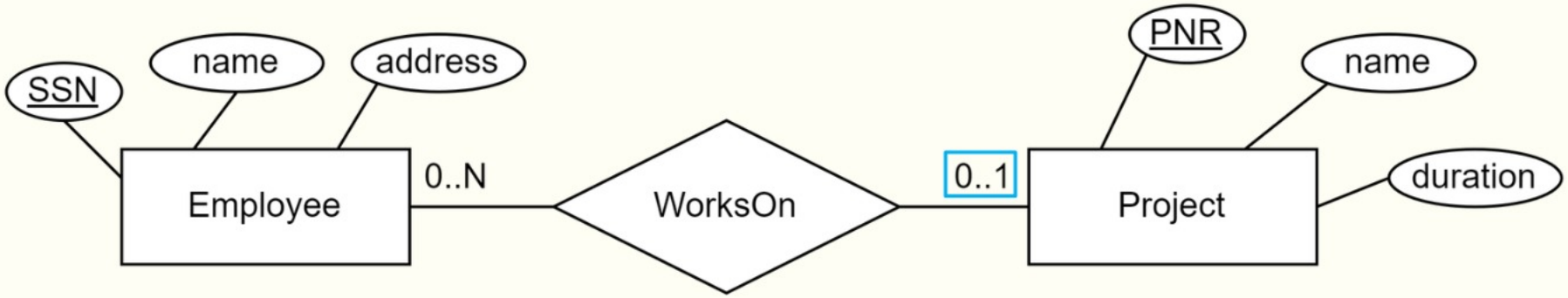
Maximum 1



- An Employee can work on at most one Project for a set number of hours

```
CREATE TABLE WorksOn (
   EmployeeID INT REFERENCES Employee (SSN),
   ProjectID INT NOT NULL REFERENCES Project (PNR),
   hours INT NOT NULL,
   PRIMARY KEY (EmployeeID) -- each Employee only appears once
);
```

Maximum 1 - No attributes in relationship

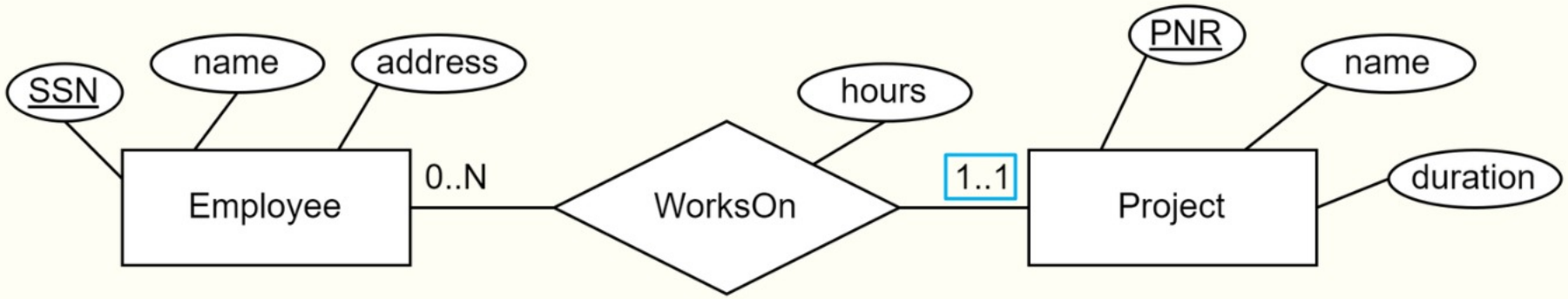


- Employee can only work on at most one Project

```
CREATE TABLE Employee (
   SSN INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   address VARCHAR(100) NOT NULL,
   PNR INT REFERENCES Project (PNR) -- WorksOn 0..1 (allows it to be NULL)
);
```

Why not do this when we have attributes?

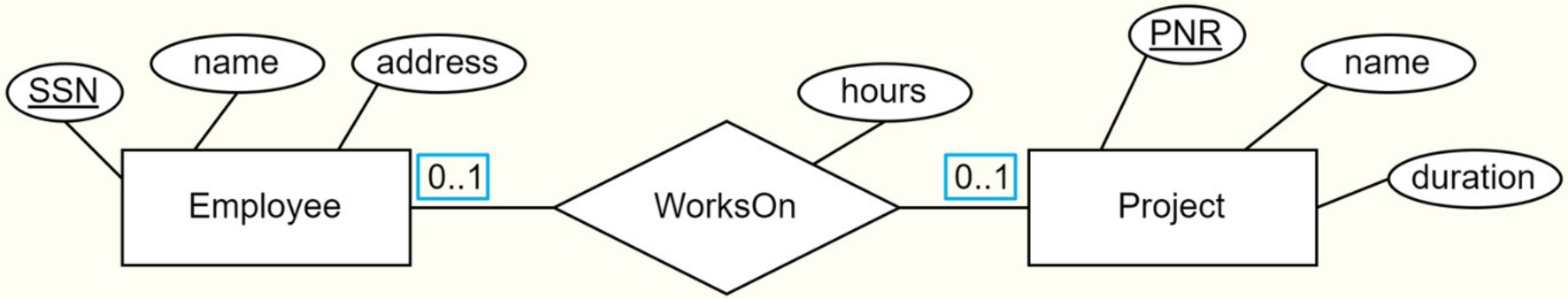
Exactly 1



- An Employee has to work on exactly one Project

```
CREATE TABLE Employee (
   SSN INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   address VARCHAR(100) NOT NULL,
   PNR INT NOT NULL REFERENCES Project (PNR), -- WorksOn 1..1
   hours INT NOT NULL -- WorksOn 1..1
);
```

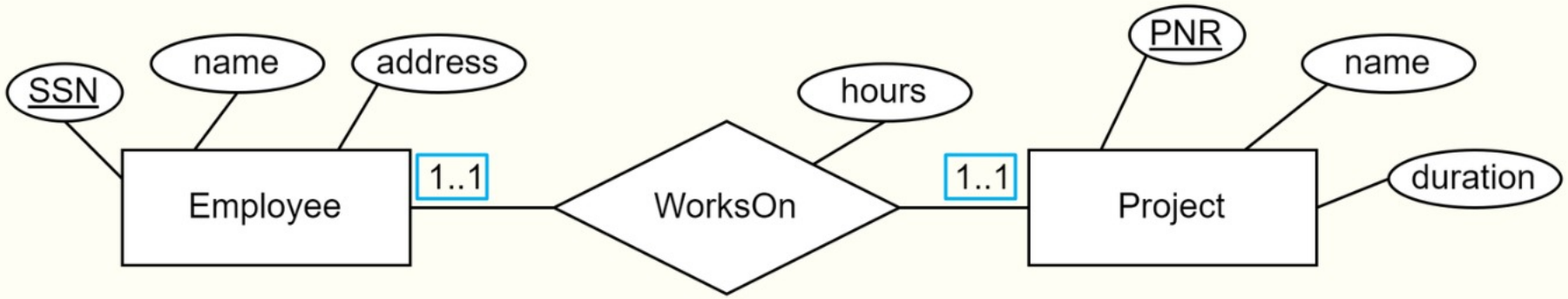
Maximum 1 - Both directions




```
CREATE TABLE WorksOn (
   EmployeeID INT REFERENCES Employee (SSN),
   ProjectID INT NOT NULL REFERENCES Project (PNR),
   hours INT NOT NULL,
   PRIMARY KEY (EmployeeID), -- ensures each Employee once
   UNIQUE (ProjectID) -- ensures each Project once
);
```

Tip: You can set the PRIMARY KEY and UNIQUE properties when declaring the attributes

Exactly 1 - Both directions



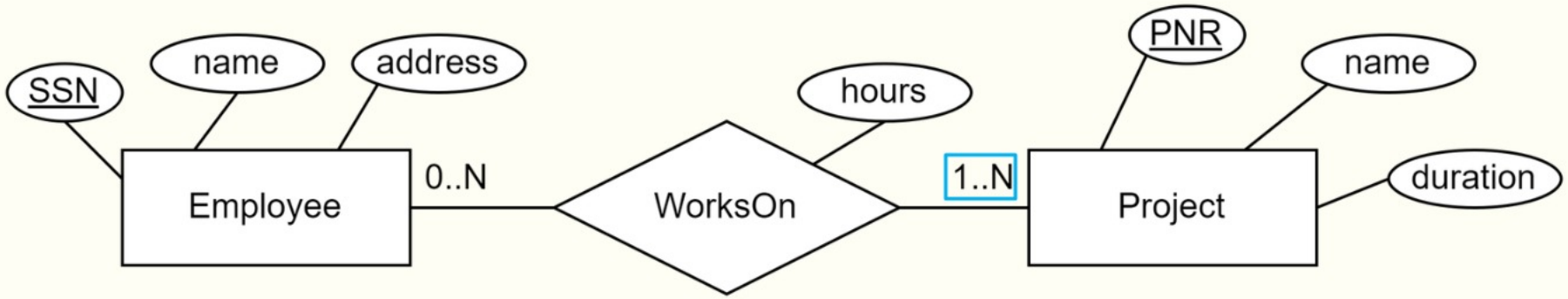

```sql
CREATE TABLE Employee (
   SSN INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   address VARCHAR(100) NOT NULL,
   PNR INT NOT NULL
   REFERENCES Project (PNR),
   hours INT NOT NULL
);
```

```sql
CREATE TABLE Project (
   PNR INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   duration INT NOT NULL,
   SSN INT NOT NULL
   REFERENCES Employee (SSN),
);
```

Exactly 1 - Both Directions

- Can we use FK on both sides?
  - Yes… but it is neither easy nor portable
- Think about inserting the first Employee and Project record
  - Which comes first… Chicken or the egg?
- Alternative 1: Use deferred FK or Trigger
  - Runs at the end of a transaction - many systems support neither!
- Alternative 2: Merge Tables
  - May work well in some cases, depends on entities
- Alternative 3: Pick one FK direction
  - Write down the other requirement
  - Do the best in software to maintain the relationship

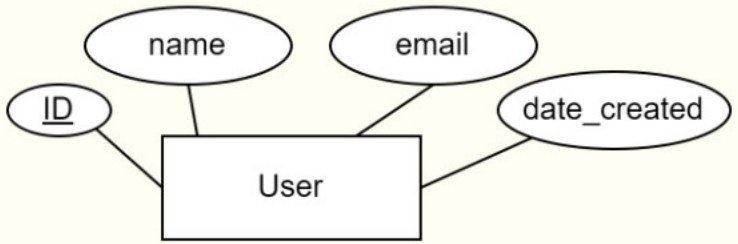
Minimum 1 - Maximum N



- An Employee works on at least one Project
- How about 1..N/M/* cardinalities?
  - Or when requirements demand a fixed number?
- No support in SQL DDL
- Normally
  - Write down the requirement
  - Do the best to maintain in software

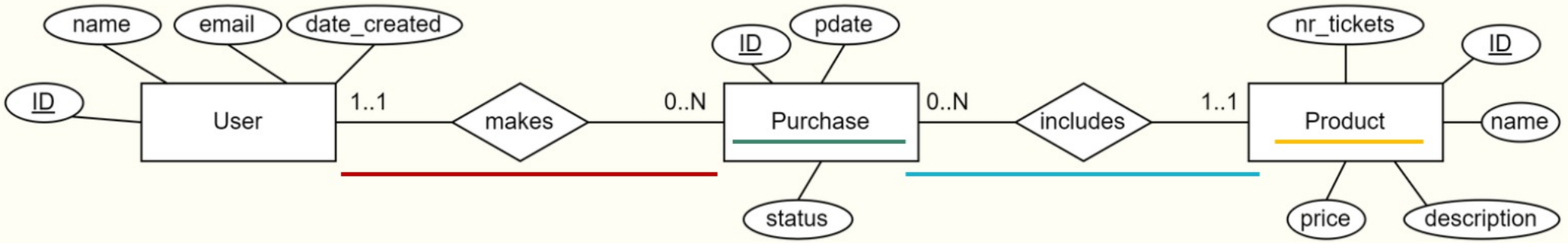
Practice (3 min)

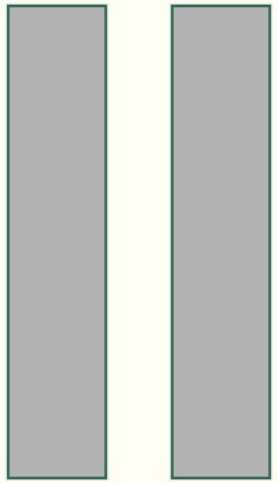
- We are given a few more requirements for our app:
  - Users can make multiple purchases, and a purchase belongs to exactly one user
  - A purchase has an ID, date of the purchase, and a status to reflect its state
  - A purchase includes exactly one product, where a product has an ID, name, description, number of tickets, and a price
  - A product can be included in different purchases
- Extend your ER model to reflect the new requirements



Practice (3 min)

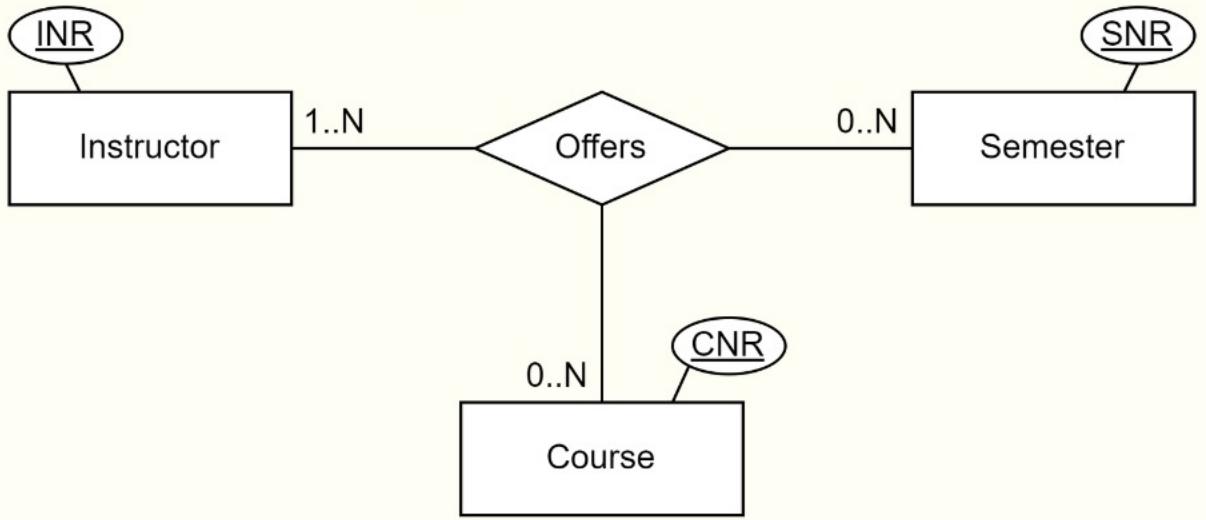
- We are given a few more requirements for our app:
  - Users can make multiple purchases, and a purchase belongs to exactly one user
  - A purchase has an ID, date of the purchase, and a status to reflect its state
  - A purchase includes exactly one product, where a product has an ID, name, description, number of tickets, and a price
  - A product can be included in different purchases
- Extend your ER model to reflect the new requirements





Ternary Relationships (and beyond)

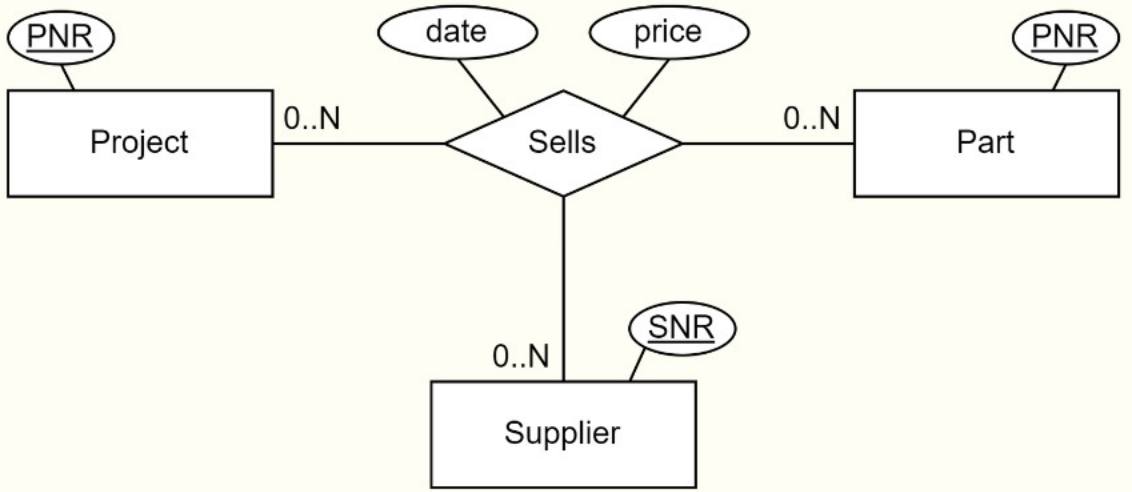
- Relationships are not limited to only two entities



- Read in the following manner:
  - Given a(n) <entity 1> and a(n) <entity 2> <relationship> <cardinality> <entity 3>
  - This can be worded differently, but the essence is the same:
    - A course being offered during a semester can have one or more instructors
    - An instructor giving a course can be offered in multiple semesters
    - During a semester, an instructor can give many courses

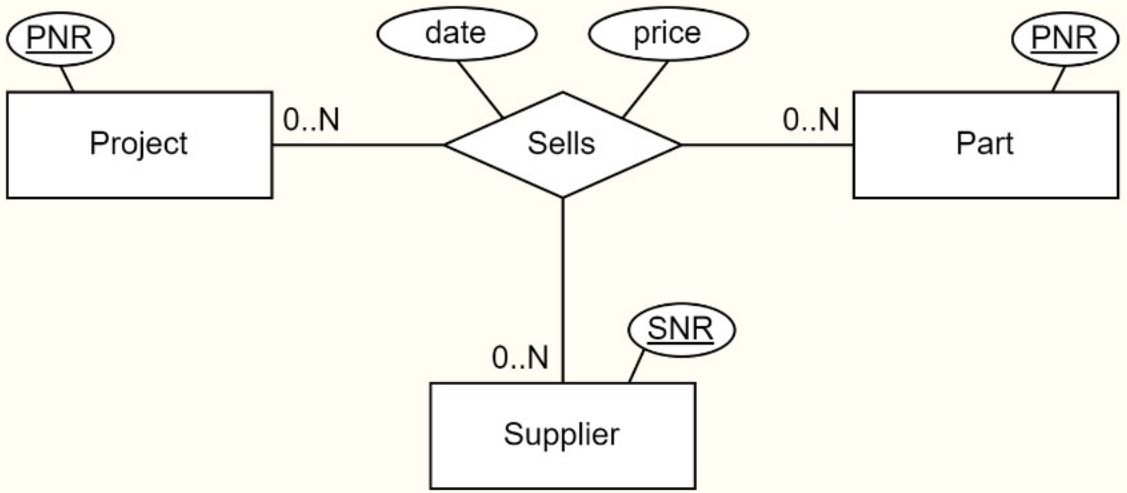
Example: Ternary Relationship DDL

```
CREATE TABLE Sells (
   Proj_nr INT REFERENCES Project (PNR),
   Supp_nr INT REFERENCES Supplier (SNR),
   Part_nr INT REFERENCES Part (PNR),
   date DATE NOT NULL,
   price FLOAT NOT NULL,
   PRIMARY KEY (Proj_nr, Supp_nr, Part_nr)
);
```



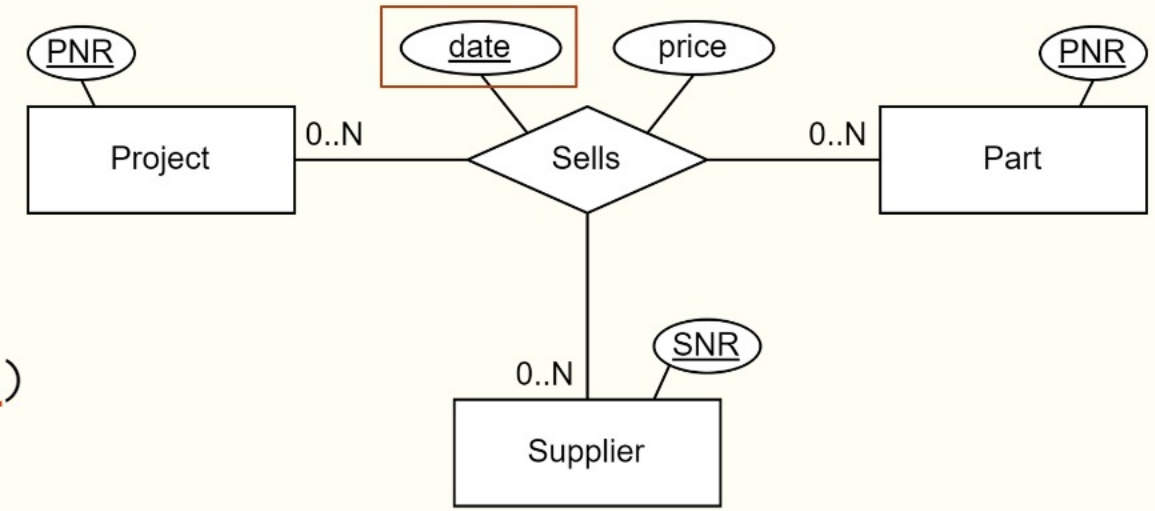
Extension: Partial Relationship Keys

- `PRIMARY KEY (Proj_nr,Supp_nr,Part_nr)`
- What does the key of the relationship table mean?
- What if the part should be sold many times?
- The book: No discussion! :(
- Our notation: Partial relationship keys
  - Attribute is underlined



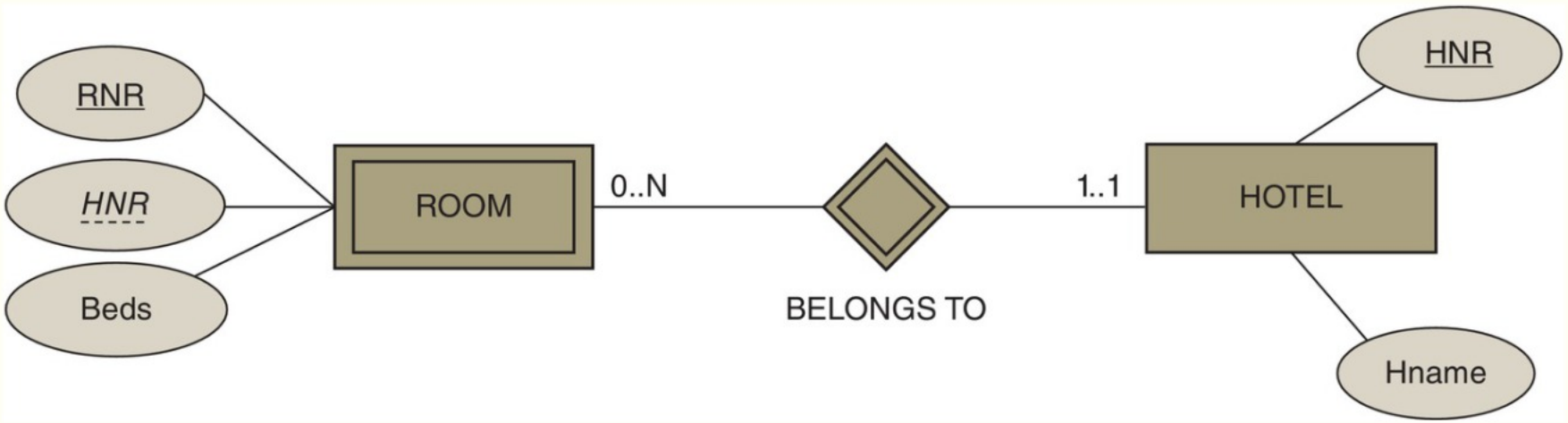
Extension: Partial Relationship Keys

```
CREATE TABLE Sells (
   Proj_nr INT REFERENCES Project (PNR),
   Supp_nr INT REFERENCES Supplier (SNR),
   Part_nr INT REFERENCES Part (PNR),
   date DATE,
   price FLOAT NOT NULL,
   PRIMARY KEY (Proj_nr, Supp_nr, Part_nr, date)
);
```



Weak Entity Types

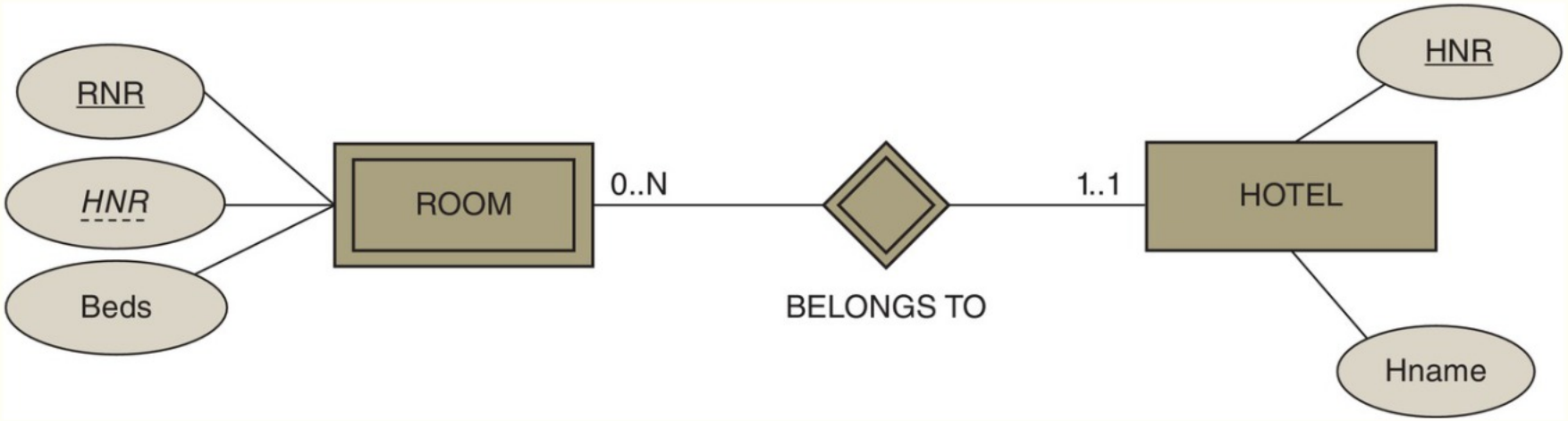
- Weak entities belong to another entity
  - They do not have a proper key, but include “parent” key
  - They have a 1 .. 1 participation in the relationship
  - If “parent” is deleted, so is the “child”
- Representation
  - Double outlines (entity, relationship)
  - Parent key is underlined with dashes



Weak Entities in SQL DDL

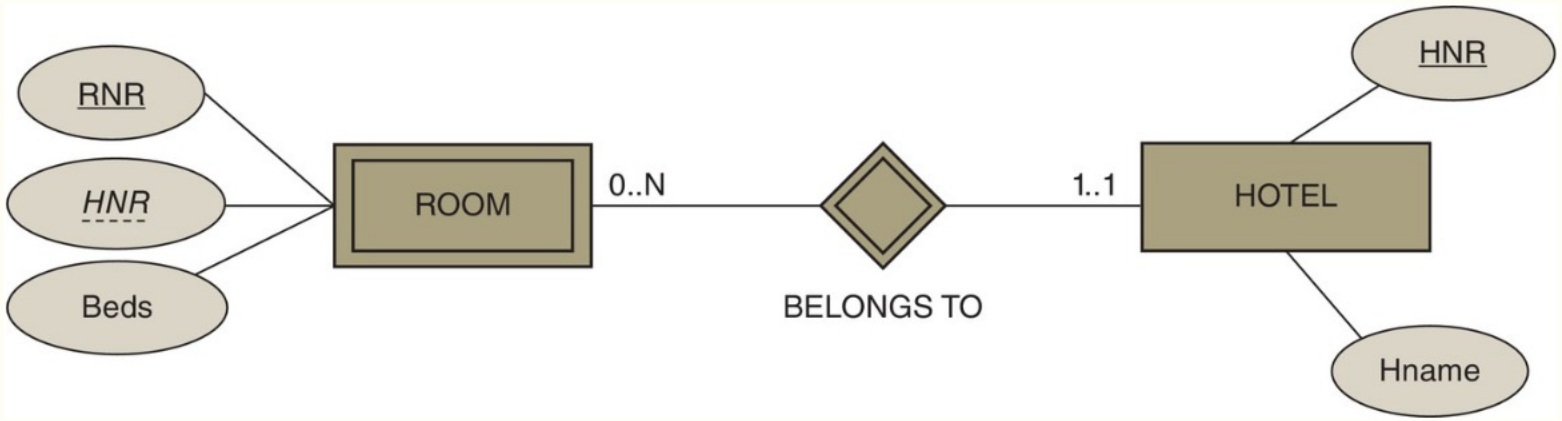
- Create a new table referencing the entity table
- Primary key is parent key + partial key
- Very similar to multi-valued attributes

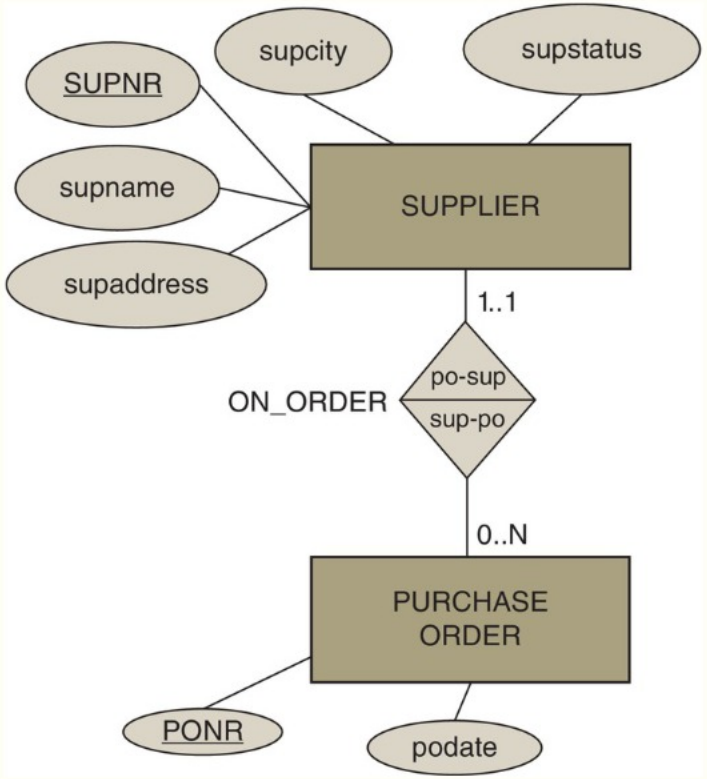
```
CREATE TABLE Room  (
   RNR INT,  -- Should not be a sequence!
   HNR INT REFERENCES Hotel (HNR),
   Beds INT NOT NULL,
   PRIMARY KEY (HNR, RNR)
);
```



Weak Entity Types vs 1..1 Relationship Types

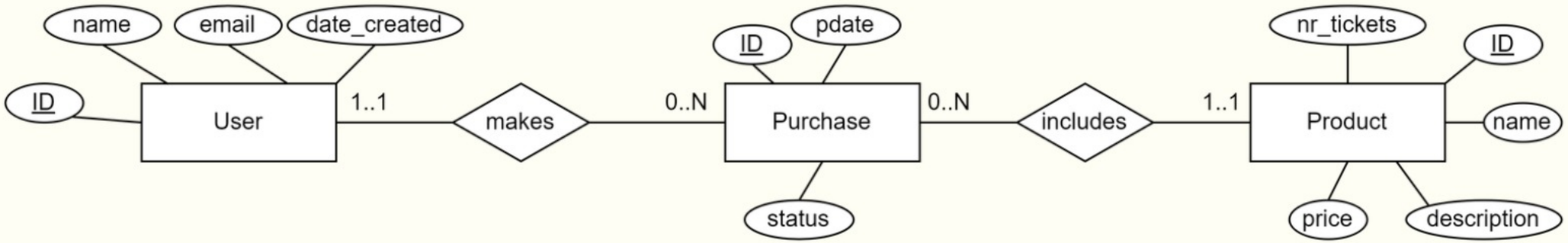
- Weak entities have a 1..1 relationship type
  - Some 1..1 relationship types represent weak entities
  - Most 1..1 relationship types do not represent weak entities
- Main difference is presence of a natural key
  - Weak entity:
    Only unique within the parent entity!





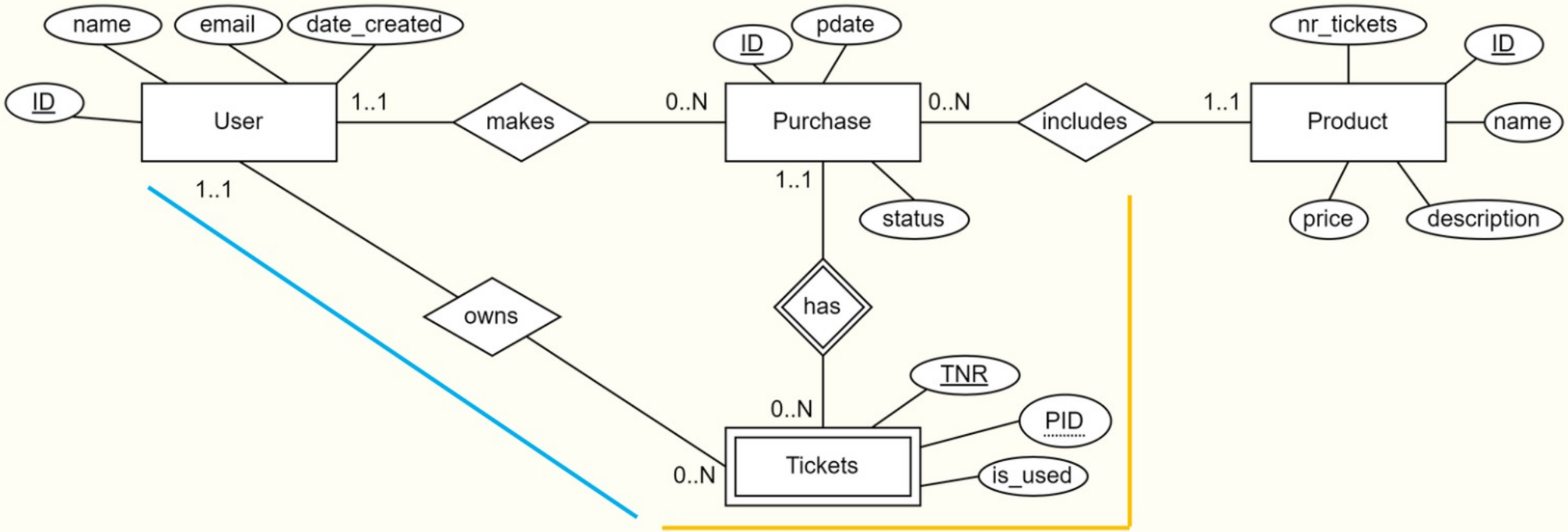
Practice (2 min)

- Let's further extend our ER model with the following requirements:
  - Every purchase has a number of tickets associated with it starting from 1 and up
  - A ticket has a number, and an indicator for whether it has been used or not
  - A ticket is owned by exactly one user



Practice (2 min)

- Let's further extend our ER model with the following requirements:
  - Every purchase has a number of tickets associated with it starting from 1 and up
  - A ticket has a number, and an indicator for whether it has been used or not
  - A ticket is owned by exactly one user

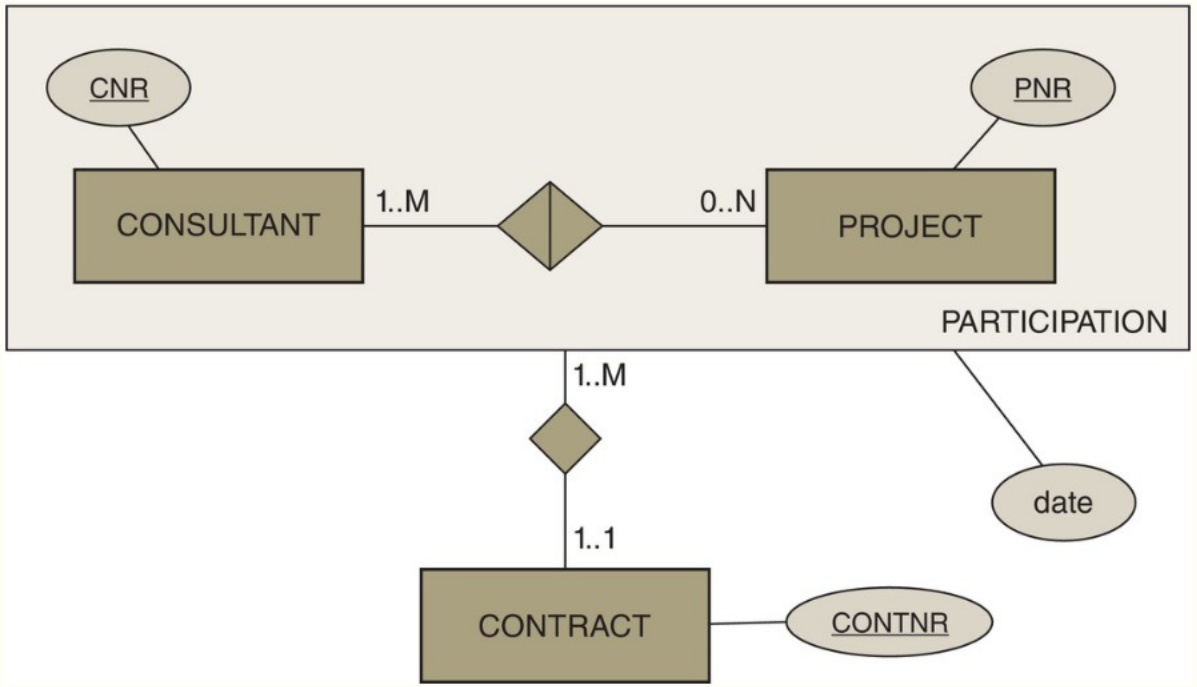


TODO

- ✓ Entity-Relationship (ER) Models / Diagrams
  - ✓ Entities and Attributes
  - ✓ Relationships
  - ✓ Cardinalities
  - ✓ Partial Relationship Keys
  - ✓ Weak Entities
- (Enhanced) ER Diagram
  - Aggregation
  - Generalization/Specialization
  - Categorization
- Translation to SQL DDL

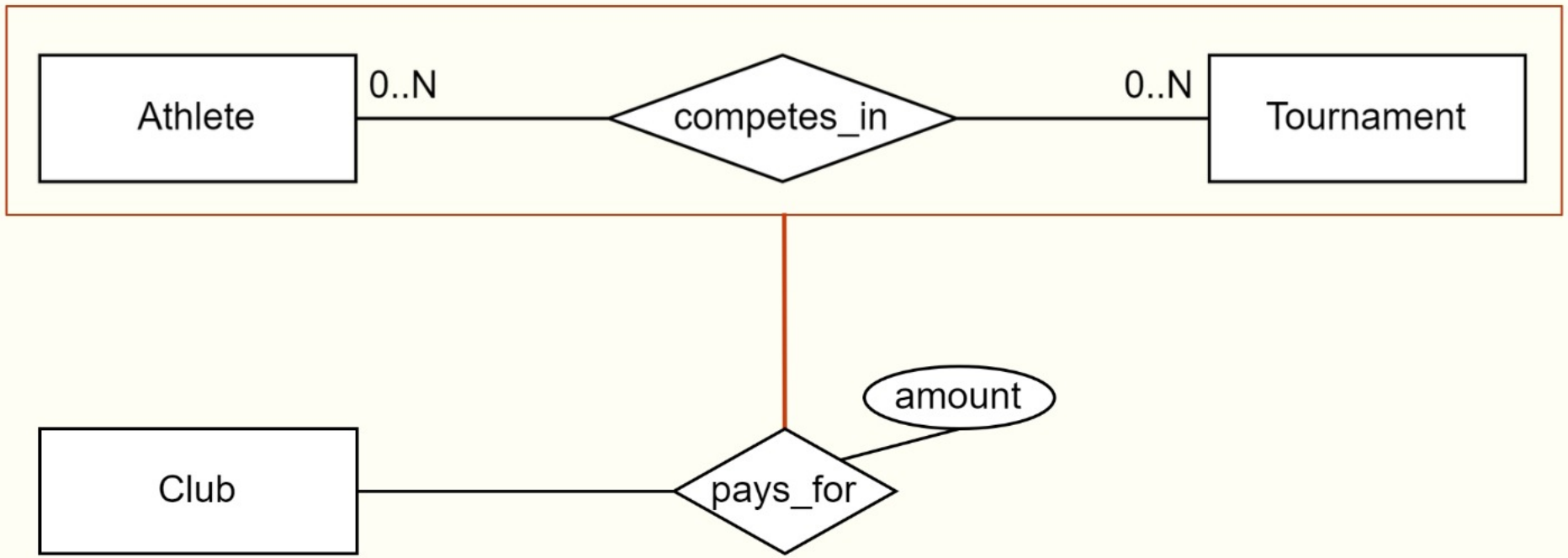
Aggregation

- Sometimes we need a relationship to a relationship
- Aggregation allows us to “convert” relationship types to entity types!
- Typical use: Monitoring, payments, contracts



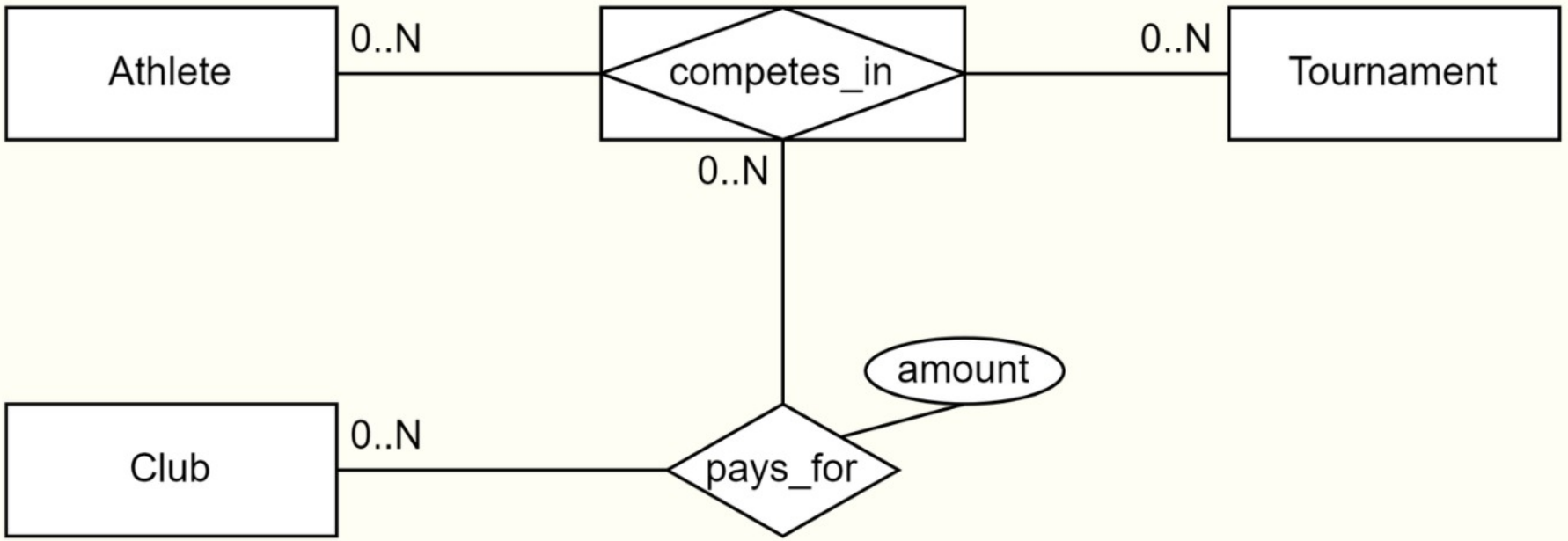
Relationship => Entity

- Consider the following ER model
  - New requirement: Clubs pay an amount to athletes competing in tournaments



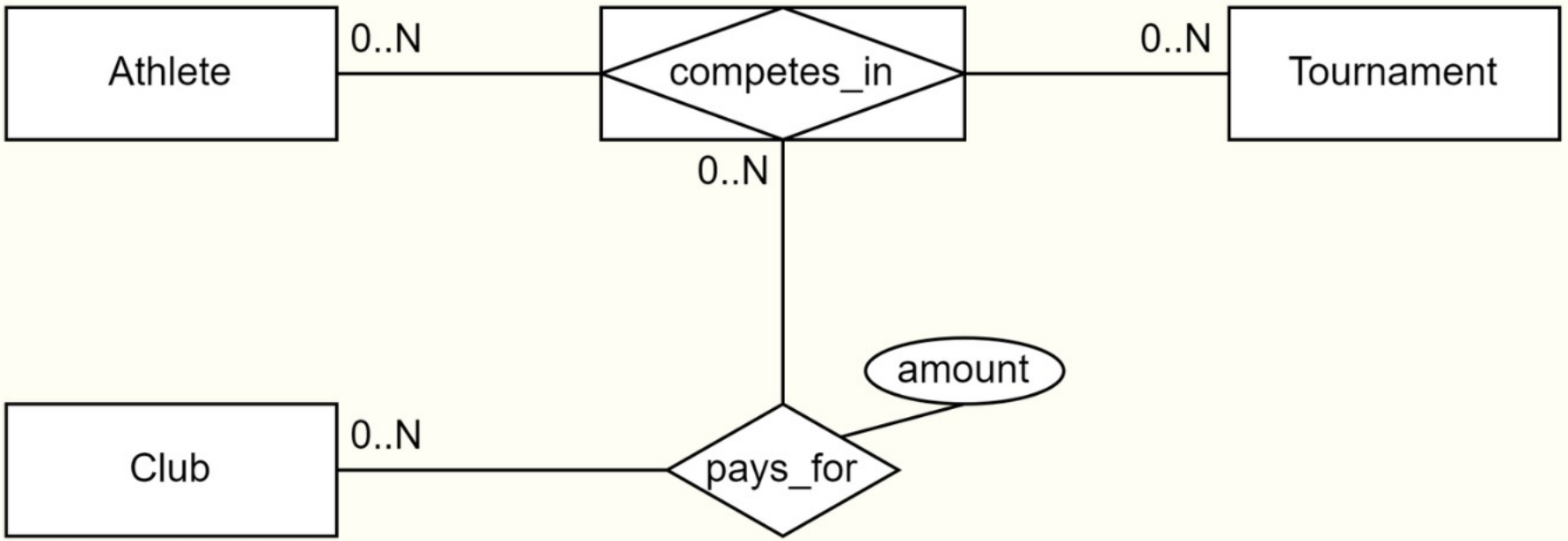
Relationship => Entity

- Consider the following ER model
  - New requirement: Clubs pay an amount to athletes competing in tournaments
  - Another way of drawing aggregation (extension)
  - We advise that you use this notation in this course, instead of the book's



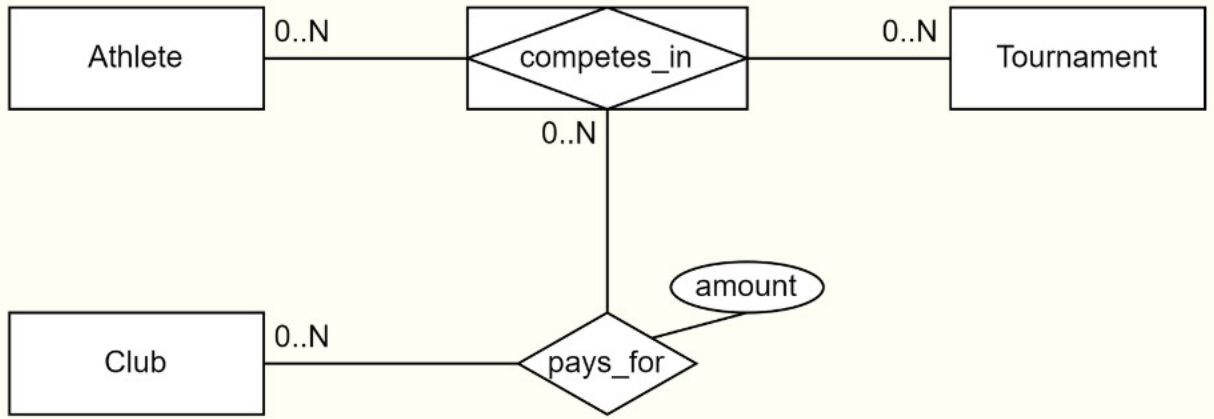
Translation to SQL DDL

- Main observation: It is a relationship!
  - Use same method as translating relationship
  - Treat the aggregation table as the entity



DDL Option 1: Use existing relationship key

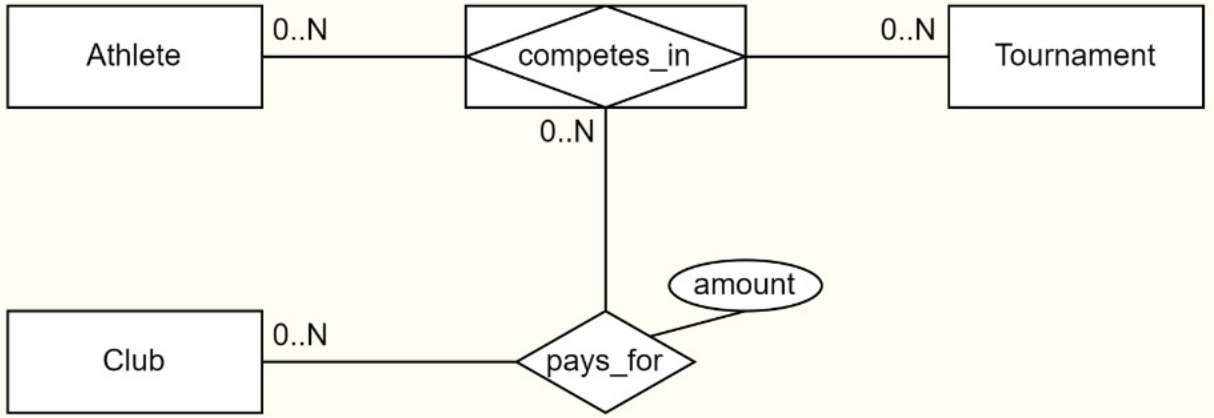
```
CREATE TABLE CompetesIn (
   AID INT REFERENCES Athletes (AID),
   TID INT REFERENCES Tournament (TID),
   PRIMARY KEY (AID, TID)
);

CREATE TABLE PaysFor (
   CID INT REFERENCES Club (CID),
   AID INT,
   TID INT,
   amount FLOAT NOT NULL,
   PRIMARY KEY (CID, AID, TID)
   FOREIGN KEY (AID, TID) REFERENCES CompetesIn (AID, TID)
);
```

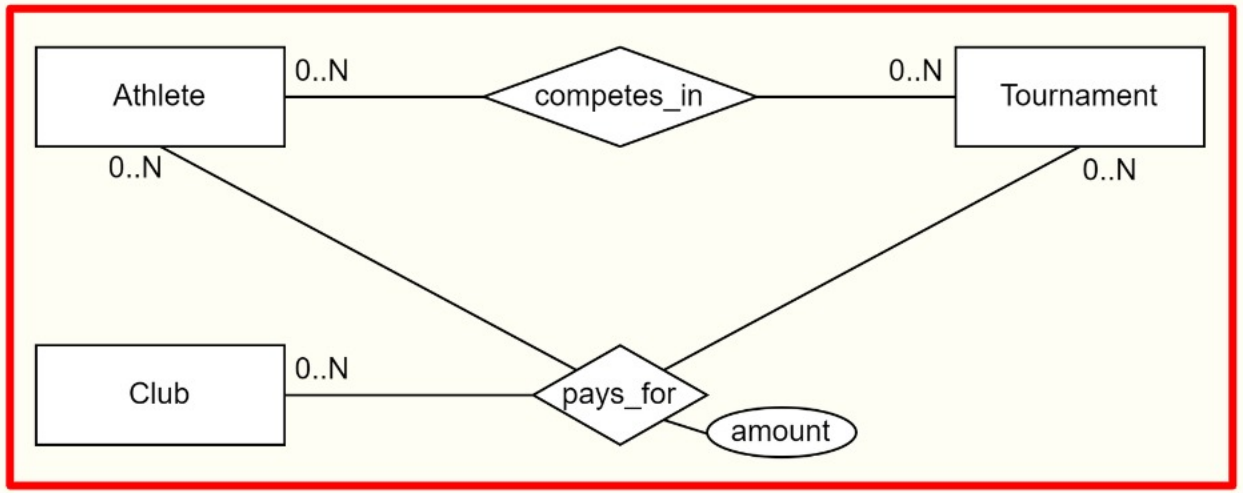


DDL Option 1: Use existing relationship key

- **Common mistake is to use FK to Entities**

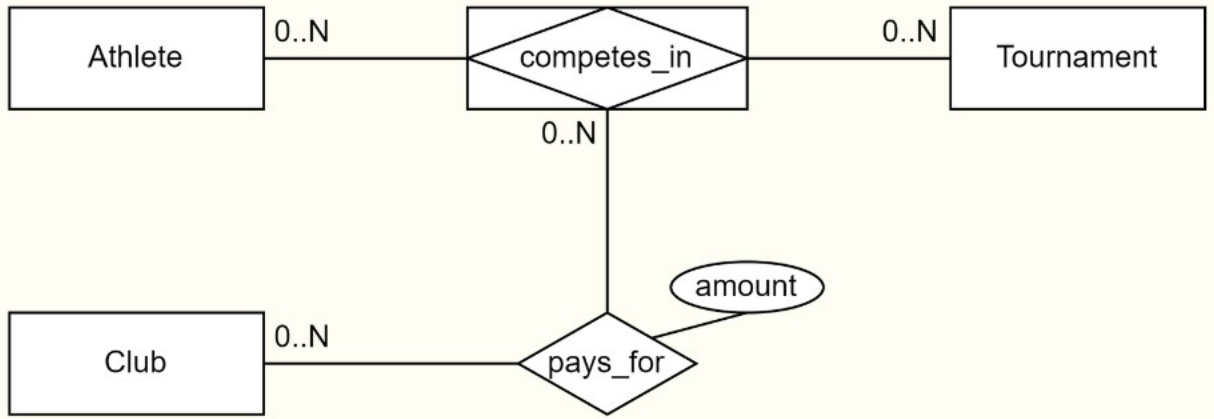
```
CREATE TABLE CompetesIn (
   AID INT REFERENCES Athletes (AID),
   TID INT REFERENCES Tournament (TID),
   PRIMARY KEY (AID, TID)
);

CREATE TABLE PaysFor (
   CID INT REFERENCES Club (CID),
   AID INT REFERENCES Athletes (AID),
   TID INT REFERENCES Tournament (TID),
   amount FLOAT NOT NULL,
   PRIMARY KEY (CID, AID, TID)
);
```





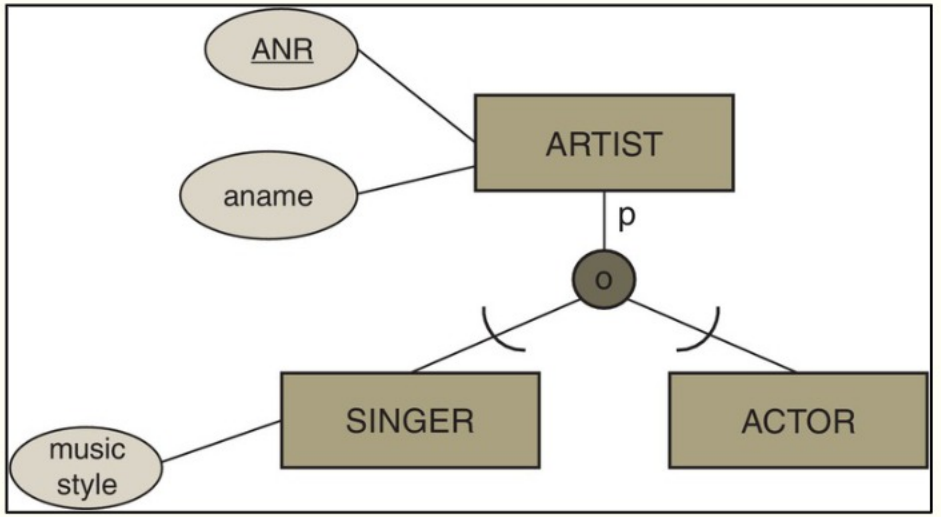
DDL Option 2: Create new relationship key

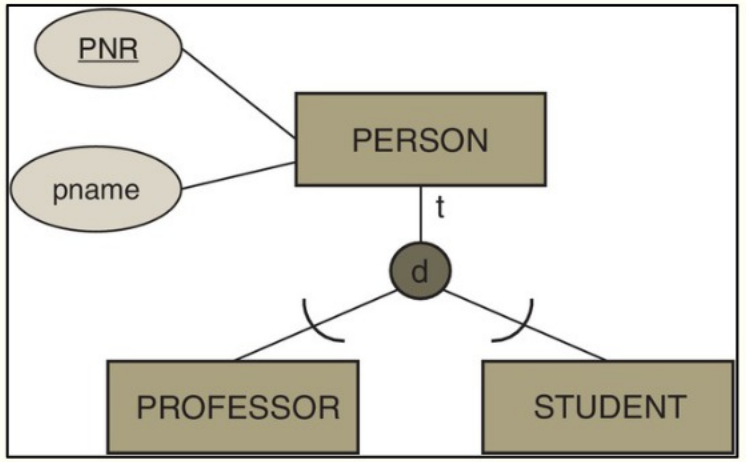
```
CREATE TABLE CompetesIn (
   CIID INT PRIMARY KEY,
   AID INT REFERENCES Athletes (AID),
   TID INT REFERENCES Tournament (TID),
   UNIQUE(AID, TID)
);

CREATE TABLE PaysFor (
   CID INT REFERENCES Club (CID),
   CIID INT REFERENCES CompetesIn (CIID),
   amount FLOAT NOT NULL,
   PRIMARY KEY (CID, CIID)
);
```



Specialization/Generalization

- Like inheritance in OOP (Java/C#/etc.)
- Partial vs Total = p/t on the line
- Overlapping vs Disjoint = o/d in the circle
- Please don't use color
- Arcs matter → need to draw them





Specialization in SQL DDL

- One table for super-type, one per sub-type
  - The PK of supertype is also PK for all subtypes
  - Each subtype has a FK to the supertype
  - Redundancy is eliminated
  - Name and DOB are stored only once
  - Adjusts well to hierarchies/lattices

Example DDL of Employee:

```
CREATE TABLE Employee (
   SSN INT PRIMARY KEY REFERENCES Person,
   Department VARCHAR(20) NOT NULL,
   Salary INT NOT NULL
);
```

Person

SSN	*Name*	*DOB*
1234	Mary	1950

Employee

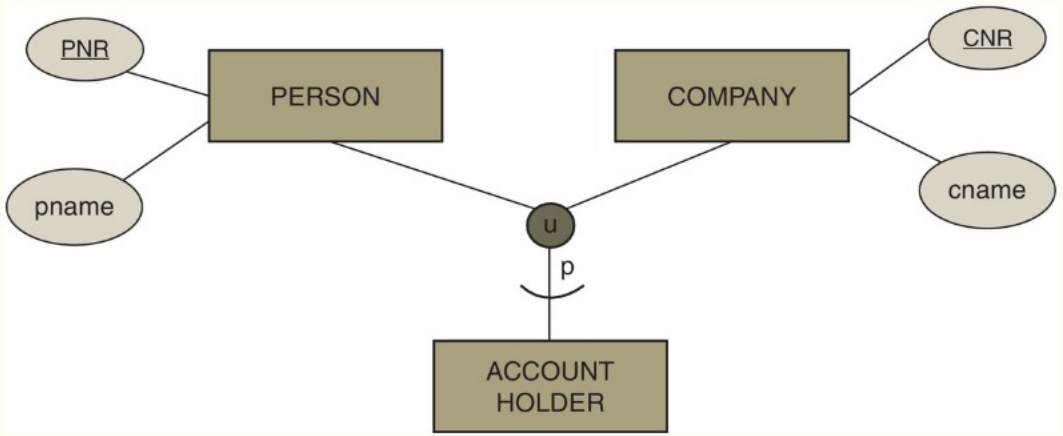
SSN	*Department*	*Salary*
1234	Accounting	35000

Student

SSN	*GPA*	*StartDate*
1234	3.5	1997

Categorization

- Grouping of otherwise unrelated entities
- New entity is a union = u in the circle
  - Can be total or partial = p/t on the line
- Notice the direction of the arc to the union
  - Refers to flow of “inheritance” – or which comes first…
  - Arcs matter → need to draw them

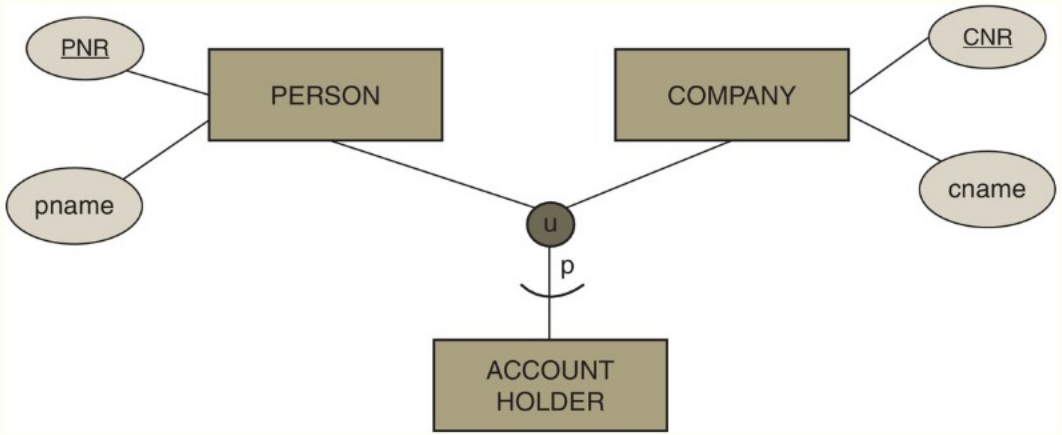


Categorization in SQL DDL

- New table for the categorical entity
  - New abstract PK with INTEGER / IDENTITY
- Add the PK as attribute to the other entities
  - With FK to the categorical entity
- Why is this important?
  - Sometimes AccountHolder participates in relationships…

```
CREATE TABLE AccountHolder (
   AcctID INT PRIMARY KEY
);

CREATE TABLE Person (
   PNR INT PRIMARY KEY,
   pname VARCHAR NOT NULL,
   AcctID INT [NOT NULL]
   REFERENCES AccountHolder (AcctID)
);
```

NOT NULL = Total
NULL = Partial



TODO

- ✓ Entity-Relationship (ER) Models / Diagrams
  - ✓ Entities and Attributes
  - ✓ Relationships
  - ✓ Cardinalities
  - ✓ Partial Relationship Keys
  - ✓ Weak Entities
- ✓ (Enhanced) ER Diagram
  - ✓ Aggregation
  - ✓ Generalization/Specialization
  - ✓ Categorization
- ✓ Translation to SQL DDL

Requirements to ER Model

- Nouns = entities
  - Descriptive elements = attributes
- Verbs = relationships
  - Descriptive elements = attributes
  - Look for words implying participation constraints
  - No words → 0..N
- Example:
  - Professors have an SSN, a name, an age, a rank, and a research specialty.
  - Projects have a project number, a sponsor name (e.g., NSF), a starting date, ...
  - Each project is managed by one professor (1..1 on profs, 0..N on projs)
  - Professors may work on many projects (0..* on both sides)
  - Each project must be reviewed by some professors (1..N on profs, 0..N on projs)

Dealing with very large ER Models

- Method 1: Very large paper!
  - One diagram with all the details
- Method 2: Outline + Details
  - One diagram with main entities and their relationships
  - One diagram per entity with attributes and weak entities
- Method 3: Components
  - Break the model into components
  - Details inside components
  - Some edge entities are repeated (details in one place)

Limitations of ER Design

- ER models do not capture all design details
  - Example: Multiple candidate keys
  - Must note missing details somewhere!
- Some aspects do not map well to SQL DDL
  - Example: 1..M cardinalities
  - Normalization (Lecture 3) provides a mechanism for fixing some problems
  - Some must simply be noted and addressed in code or ignored!

Takeaways!

- An ER model captures entities and relationships
  - Weak entities, specialization, categorization and aggregation allow capturing more detailed model characteristics
  - This is hard – but also useful – so you must practice!
- Conversion to SQL DDL
  - Entities and relationships mapped to relations
  - Essentially an algorithmic process (with some options)
  - This is hard – but also useful – so you must practice!
- Notation: MANY VARIANTS EXIST!
  - We use the one from the book (as extended in lecture)
  - You must use this notation in the homework and exam!!!